

practical-1

AIM-write a program to read your name contact number ,email and birthday AND print those detail on the screen.

```
# taking input form user
```

```
hour_worked=float(input("enter the number:"))
hourly_rate=float(input("enter hourly rate:"))

if hour_worked > 40:
    overtime=hour_worked-40
    total_wages=40*hourly_rate+1.5*overtime*hourly_rate
else:
    total_wages=hour_worked*hourly_rate
    print("totalwages in week",total_wages)
```

```
enter the number:10
enter hourly rate:1000
totalwages in week 10000.0
```

```
2type(a)
```

```
str
```

```
print(name)
```

```
vATSAL
```

Start coding or [generate](#) with AI.

```
name=input("enter name:")
number=input("enter your number:")
email=input("enter your email")
birthdate=input("enter your birthday")
print(name,number,email,birthdate)
```

```
enter name:vatsal
enter your number:9173018192
enter your emailvatsalmodi23@gmail.ocm
enter your birthday23-9-2005
vatsal 9173018192 vatsalmodi23@gmail.ocm 23-9-2005
```

practical-2

AIM: Write a program to convert temperature from Celsius to Fahrenheit. Equation to convert Celsius to Fahrenheit: $F = (9/5) * C + 32$

```
celsius = float(input("enter temperature in celsius no: "))
fahrenheit = (9/5)*celsius+32

print(fahrenheit)
```

```
enter temperature in celsius no: 50
122.0
```

```
fahrenheit=float(input("enter the fahrenheit no:"))
celsius=(fahrenheit-32)*5/9

print(celsius)
```

```
enter the fahrenheit no:12
-11.111111111111111
```

Start coding or [generate](#) with AI.

practical-3

AIM: Write a program to compute the slope of a line between two points (x1, y1) and (x2, y2). $Slope = (y_2 - y_1) / (x_2 - x_1)$

```
x1=23
x2=25
y1=65
y2=85
slope = ( y2 - y1 )/(x2 - x1)
midpoint_x = x1+x2/2
midpoint_y = y1+y2/2
print("slope of line: ")
print(slope)
```

```
slope of line:
10.0
```

practical-4

AIM: Write a program to calculate simple and compound interest. $Simple\ Interest = (P * R * T) / (100)$ $Compound\ Interest = principle * (pow((1 + rate / 100), time))$

```
principle=5000
rate= 15.11
time = 15.11
n = 3
compound_interest = principle * (1 + rate / (100 * n))**(n * time)
print("Simple Interest", principle * rate * time/100)
print("Compound interest is", compound_interest)
```

```
Simple Interest 11415.605
Compound interest is 46382.526989207574
```

Start coding or [generate](#) with AI.

practical-5

AIM: Write a program to get change values in Quarter, Dime, Nickels and Pennies, and calculate the value of change in Dollars. Consider Quarter = 0.25 ,Dime=0.10 , Nickels = 0.05 and Penny=0.01 .

```
quarter = int(input("enter number:"))
dime = int(input("enter number:"))
nickels = int(input("enter number:"))
penny = int(input("enter number:"))
```

```
total = 25*quarter + 10*dime + 5*nickels + penny
dollars = total/100.0
print(dollars)
```

```
quarter=int(input("enter number:"))
dime=int(input("enter number:"))
nickels=int(input("enter number:"))

penny=int(input("enter number:"))

total=25*quarter+10*dime+5*nickels+pennyj
dollars=total/100.00
print(dollars)
```

practical-6

AIM: Write a program to find a maximum of given three numbers (Use ternary operator).

```
a = int(input("Enter no:"))
b = int(input("Enter no:"))
c = int(input("Enter no:"))

max = (a if (a > b and a > c) else
      (b if (b > a and b > c) else c))
print("largest number no: ",max )
```

```
Enter no:10
Enter no:20
Enter no:30
largest number no: 30
```

practical-7

AIM: Write a program to calculate area and volume of Sphere. *Area of Sphere* = $4\pi r^2$ *Volume of Sphere* = $\frac{4}{3}\pi r^3$

```
import math
r=float(input("Enter the r of a sphere"))
area=4*math.pi*pow(r,2)
volume=(4/3)*math.pi*pow(r,3)

print("surface area:")
print(area)
print("volume:")
print(volume)
```

```
Enter the r of a sphere12
surface area:
1809.5573684677208
volume:
7238.229473870882
```

Start coding or [generate](#) with AI.

practical-8

AIM: Write a program that computes the real roots of a given quadratic equation (Use math library). Discriminant $\Delta = b^2 - 4ac$

RealRoots = $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

```
import math
a=int(input("Enter the value of a: "))
b=int(input("Enter the value of b: "))
c=int(input("Enter the value of c: "))
d=(b*b)-(4*a*c)
print(d)
if d>0:
    root1=(-b-math.sqrt(d))/(2*a)
    root2=(-b+math.sqrt(d))/(2*a)
    print("The roots of equation are",root1)
    print("The roots of equation are",root2)
else:
    print("This equation has no real roots")
```

```
Enter the value of a: 3
Enter the value of b: 5
Enter the value of c: 7
-59
This equation has no real roots
```

Start coding or [generate](#) with AI.

practical-9

write a program to determine the length of ladder required to reach a given height when leaned against the house. the height and the angle of the ladder are given as input (use math library)

```
height = int(input("Enter the height of wall: "))
angle = int(input("Enter the angle the ladder: "))
length = height/math.sin(math.radians(angle))
print(length)
```

```
Enter the height of wall: 12
Enter the angle the ladder: 14
49.60278593326499
```

practical-10

Aim: A year is a Leap year if it is divisible by 4, unless it is a century year that is not divisible by 400 (1800 and 1900 are not leap years, 1600 and 2000 are leap years). Write a program that calculates whether a given year is a leap year or not.

```

year = int(input("enter a year: "))

if (year % 400 == 0) and (year % 100 == 0):
    print("{0} is a leap year".format(year))

elif (year % 4 == 0) and (year % 100 != 0):
    print("{0} is a leap year".format(year))

else:
    print("{0} is not a leap year".format(year))

```

```

enter a year: 2016
2016 is a leap year

```

practical-11

Many companies pay time-and-a-half for any hours worked above 40 hours in a given week. Write a program to input the number of hours worked and hourly rate and calculate the total wages for the week.

```

hours_worked = float(input('Enter the number of hours worked: '))
hourly_rate = float(input('Enter the hourly rate: '))

if hours_worked > 40:
    overtime = hours_worked - 40
    total_wages = 40 * hourly_rate + 1.5 * overtime * hourly_rate
else:
    total_wages = hours_worked * hourly_rate

print('The total wages for the week are: ', total_wages)

```

```

Enter the number of hours worked: 45
Enter the hourly rate: 300
The total wages for the week are: 14250.0

```

Double-click (or enter) to edit

```

l="hiren"
print(l[2],l[3])

```

```

r e

```

```

l=[0,1,2,3,4]
for i in range(len(l)):
    l[i]=l[i]**2
print(l)

```

```

[0, 1, 4, 9, 16]

```

practical-12

The Body Mass Index is calculated as a person's height(in meters). If the BMI is between 19 and 25, the person is healthy. If the BMI is below 19, then the person is underweight. If the BMI is above 25, then the person is overweight. Write a program to get a person's weight(in kgs) nad height(in cms) and display a message whether the person is healthy, underweight or overweight. BMI= weight in kg/(height in m)²

```

weight = float(input('enter the weight:'))
height = float(input('enter the height:'))
BMI = int(weight/(height/100)**2)
if BMI in range(19,25):
    print('person is healthy')
elif(BMI<19):
    print('person is underweight')
elif(BMI>25):
    print('person is overweight')
else:
    print('invalid BMI')

```

```

enter the weight:50
enter the height:150
person is healthy

```

practical 13: write a program to read the marks and assign a grade to a student. grading system : a(>=90), b(80-89), c(70-79), d(60-69), e(50-59), f(< 50) (use the switch case)

```

marks = int(input("enter marks: "))
def grade(marks):
    if (marks>=90):
        print("A grade")
    elif(marks in range(80,89)):
        print("B grade")
    elif(marks in range(70,79)):
        print("C grade")
    elif(marks in range(60,69)):
        print("D grade")
    elif(marks in range(50,59)):
        print("E grade")
    else:
        print("none")

```

```

grade(marks)

```

```

enter marks: 95
A grade

```

practical:14

write a program to read n number from users and calculate the average of those n number.

```

def average(*args):
    c = args[0:]
    print(sum(c))
    print(len(c))
    avg = sum(c)/ len(c)
    return avg
average(1,4,5,7,10)

```

```
27
5
5.4
```

practical:15

write a program to print below a pattern:

```
*
```

```
•   °
```

```
def pattern(rows):
    k = 2 * rows - 2
    for i in range(0, rows):
        for j in range(0, k):
            print(end=" ")
        k = k - 1
        for j in range(0, i + 1):
            print("* ", end="")
        print("")
    pattern(6)
```

```
      *
     **
    ***
   ****
  *****
 *****
*****
```

practical:16 write a program to print below a pattern:

112123123412345

```
def pattern (rows):
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(j, end=' ')
        print('')
    pattern(5)
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Start coding or [generate](#) with AI.practical:17 $1/3 + 3/5 + 5/7 +$

```
def series():
    sum = 0
    for i in range(1,100,2):
        sum += i/(i+2)
    print("Sum of the Series = ",sum)
series()
```

Sum of the Series = 46.10464832285218

Practical :-18

```
# Function to display ASCII character table
def display_ascii_table():
    start_char = 33 # ASCII value for '!'
    end_char = 126 # ASCII value for '~'
    chars_per_line = 5

    print(f"{'Char':^5} {'Dec':^5} {'Hex':^5} " * chars_per_line)
    print('-' * 21 * chars_per_line)

    for i in range(start_char, end_char + 1):
        char = chr(i)
        dec_val = i
        hex_val = hex(i)

        print(f"{char:^5} {dec_val:^5} {hex_val:^5}", end=' ')

        if (i - start_char + 1) % chars_per_line == 0:
            print()

    if (end_char - start_char + 1) % chars_per_line != 0:
        print()

# Call the function to display the table
display_ascii_table()
```

Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex
!	33	0x21	"	34	0x22	#	35	0x23	\$	36	0x24	%	37	0x25
&	38	0x26	'	39	0x27	(	40	0x28	)	41	0x29	*	42	0x2a
+	43	0x2b	,	44	0x2c	-	45	0x2d	.	46	0x2e	/	47	0x2f
0	48	0x30	1	49	0x31	2	50	0x32	3	51	0x33	4	52	0x34
5	53	0x35	6	54	0x36	7	55	0x37	8	56	0x38	9	57	0x39
:	58	0x3a	;	59	0x3b	<	60	0x3c	=	61	0x3d	>	62	0x3e
?	63	0x3f	@	64	0x40	A	65	0x41	B	66	0x42	C	67	0x43
D	68	0x44	E	69	0x45	F	70	0x46	G	71	0x47	H	72	0x48
I	73	0x49	J	74	0x4a	K	75	0x4b	L	76	0x4c	M	77	0x4d
N	78	0x4e	O	79	0x4f	P	80	0x50	Q	81	0x51	R	82	0x52
S	83	0x53	T	84	0x54	U	85	0x55	V	86	0x56	W	87	0x57
X	88	0x58	Y	89	0x59	Z	90	0x5a	[	91	0x5b	\	92	0x5c
]	93	0x5d	^	94	0x5e	_	95	0x5f	`	96	0x60	a	97	0x61
b	98	0x62	c	99	0x63	d	100	0x64	e	101	0x65	f	102	0x66
g	103	0x67	h	104	0x68	i	105	0x69	j	106	0x6a	k	107	0x6b
l	108	0x6c	m	109	0x6d	n	110	0x6e	o	111	0x6f	p	112	0x70
q	113	0x71	r	114	0x72	s	115	0x73	t	116	0x74	u	117	0x75
v	118	0x76	w	119	0x77	x	120	0x78	y	121	0x79	z	122	0x7a
{	123	0x7b		124	0x7c	}	125	0x7d	~	126	0x7e			

practical:19 A positive integer is called a perfect number if it is equal to the sum of all of its positive divisors, excluding itself. For example, 6 is the first perfect number, because $6 = 3 + 2 + 1$, the next is $28 = 14 + 7 + 4 + 2 + 1$. There are four perfect numbers that are less than 10,000. Write a program to find these four numbers.

```
def perfect(num):
    sum = 0
    for i in range(1,num):
        if num % i == 0:
            sum += i
    return sum == num

for n in range(1,10000):
```

```

if(perfact(n)):
    print("Perfect numbers less than 10,000:", n)

```

```

Perfect numbers less than 10,000: 6
Perfect numbers less than 10,000: 28
Perfect numbers less than 10,000: 496
Perfect numbers less than 10,000: 8128

```

practical:20 Write a program to perform the below operations on the list: □ Create a list. □ Add/Remove an item to/from a list. □ Get the number of elements in the list. □ Access elements of the list using the index. □ Sort the list. □ Reverse the list.

```

list1 = [1,2,3,4,5]

list1.append(6)
print('add number =>',list1)

list1.remove(6)
print('remove number =>',list1)

print('length of =>',len(list1))

print('update the list with accessing list',list1[2])

list1.sort()
print('sort list =>',list1)

list1.reverse()
print('reverse list =>',list1)

```

```

add number => [1, 2, 3, 4, 5, 6]
remove number => [1, 2, 3, 4, 5]
length of => 5
update the list with accessing list 3
sort list => [1, 2, 3, 4, 5]
reverse list => [5, 4, 3, 2, 1]

```

practical:21 Write a program to read n numbers from a user and print: ● Number of positive numbers. ● Number of negative numbers. ● Number of zeros. ● Number of odd numbers. ● Number of even numbers. ● Average of all numbers.

```

def analyze_numbers():
    n = int(input("Enter the number of elements: "))

    # Initialize counters and sum
    positive_count = 0
    negative_count = 0
    zero_count = 0
    odd_count = 0
    even_count = 0
    total_sum = 0

    # Read numbers and analyze them
    for i in range(n):
        number = float(input(f"Enter number {i+1}: "))
        total_sum += number

        if number > 0:
            positive_count += 1

```

```

elif number < 0:
    negative_count += 1
else:
    zero_count += 1

if number % 2 == 0:
    even_count += 1
else:
    odd_count += 1

# Calculate average
print("Sum of = ",total_sum)
average = total_sum / n

# Print results
print("Number of positive numbers: {positive_count}")
print("Number of negative numbers:", negative_count)
print(f"Number of zeros: {zero_count}")
print(f"Number of odd numbers: {odd_count}")
print(f"Number of even numbers: {even_count}")
print(f"Average of all numbers: {average}")

# Call the function
analyze_numbers()

```

```

Enter the number of elements: 5
Enter number 1: 1
Enter number 2: 2
Enter number 3: 3
Enter number 4: 4
Enter number 5: 6
Sum of = 16.0
Number of positive numbers: {positive_count}
Number of negative numbers: 0
Number of zeros: 0
Number of odd numbers: 2
Number of even numbers: 3
Average of all numbers: 3.2

```

Start coding or [generate](#) with AI.

practical-22 Write a program that counts the occurrences of each digit in a string. The program counts how many times a digit appears in the string. For example, if the input is "12203AB3", then the output should output 0 (1 time), 1 (1 time), 2 (2 times), 3 (2 times).

Double-click (or enter) to edit

```

user_input = input('enter the string')

for i in range(10):
    count = user_input.count(str(i))
    print(f"{i} occure {count} times ")

```

```

enter the string12203ab3
0 occure 1 times
1 occure 1 times
2 occure 2 times
3 occure 2 times
4 occure 0 times
5 occure 0 times
6 occure 0 times
7 occure 0 times

```

```
8 occure 0 times
9 occure 0 times
```

practical-23 Write a program to eliminate duplicate values in the list.

```
def remove_duplicates(input_list):
    # Convert the list to a set to eliminate duplicates, then convert it back
    unique_list = list(set(input_list))

    # Optionally, sort the list to maintain order
    #unique_list.sort()

    return unique_list

# Example usage
input_list = [1, 2, 3, 2, 4, 5, 1, 6, 3, 7]
print("Original List:", input_list)

unique_list = remove_duplicates(input_list)
print("List after removing duplicates:", unique_list)
```

```
Original List: [1, 2, 3, 2, 4, 5, 1, 6, 3, 7]
List after removing duplicates: [1, 2, 3, 4, 5, 6, 7]
```

```
list1 = [10,20,20,30,40,40,50]
list2=[]

for i in list1:
    if i not in list2:
        list2.append(i)
print(list2)
```

```
[10, 20, 30, 40, 50]
```

Practical :- 24

Write a program that defines a function to determine whether input number n is prime or not. A positive whole number $n > 2$ is prime, if no number between 2 and n (inclusive) evenly divides n . If n is not prime, the program should quit as soon as it finds a value that evenly divides n .

```
def is_prime(n):
    if n <= 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            print(f"{n} is not prime; divisible by {i}.")
            return False
    print(f"{n} is prime.")
    return True

# Example usage:
number = int(input("Enter a positive whole number greater than 2: "))
is_prime(number)
```

practical-25 A list contains the average daily temperature(in degree Celsius) of a city over a particular week. Write a Python code to swap the highest and the lowest temperatures. temperatures = [34, 40, 29, 33, 42, 37, 39]

```
temp = [34,40,29,33,42,37,39]

print(f"the max is:{max(temp)}")
print(f"the min is:{min(temp)}")

max_index = temp.index(max(temp))
min_index = temp.index(min(temp))

temp[max_index],temp[min_index] = temp[min_index],temp[max_index]
print(f"the list after swaped:{temp}")
```

```
the max is:42
the min is:29
the list after swaped:[34, 40, 42, 33, 29, 37, 39]
```

practical-26 Write a program to perform below operations on tuple: □ Create a tuple with different data types. □ Print tuple items. □ Convert tuple into a list. □ Remove data items from a list. □ Convert list into a tuple. □ Print tuple items.

```
Tuple = (3,"Hello",356,False)
print("Tuple:",Tuple)
print("indexing on tuple :",Tuple[1])
print("negtive indexing on tuple :",Tuple[-1])
print("slicing on tuple :",Tuple[2:5])

List = list(Tuple)
print("Convert into List:",List)
List.pop()
print("remove data from list:",List)
Tuple = tuple (List)
print("convert to a tuple:",Tuple)
```

```
Tuple: (3, 'Hello', 356, False)
indexing on tuple : Hello
negtive indexing on tuple : False
slicing on tuple : (356, False)
Convert into List: [3, 'Hello', 356, False]
remove data from list: [3, 'Hello', 356]
convert to a tuple: (3, 'Hello', 356)
```

practical-27 Write a program to perform below operations on set: □ Create two different sets with the data. □ Print set items. □ Add/remove items in/from a set. □ Perform operations on sets: union, intersection, difference, symmetric difference, check subset of another set.

Start coding or [generate](#) with AI.

```
a=set({10,20,30,40})
b=set({30,40,50,60})
print("First set:",a)
print("Second set:",b)
a.add(50)
a.remove(40)
print("after add/remove element of set",a)
x=a.union(b)
y=a.intersection(b)
```

```
print("union:",x)
print("intersection:",y)
print("Difference:",y.difference(b))
print("Symmertric Differece:",y.symmetric_difference(b))
print("Subset:",y.issubset(b))
```

```
First set: {40, 10, 20, 30}
Second set: {40, 50, 60, 30}
after add/remove element of set {10, 50, 20, 30}
union: {40, 10, 50, 20, 60, 30}
intersection: {50, 30}
Difference: set()
Symmertric Differece: {40, 60}
Subset: True
```

Practical :-28

```
s=(input('enter your data:'))
for i in range(10):
    y=s.count(str(i))
    print(i,"occurs",y,"times")
```

```
enter your data:12
0 occurs 0 times
1 occurs 1 times
2 occurs 1 times
3 occurs 0 times
4 occurs 0 times
5 occurs 0 times
6 occurs 0 times
7 occurs 0 times
8 occurs 0 times
9 occurs 0 times
```

Practical :-29

```
temp={ "sunday":40,
        "monday":39,
        "tuesday":41,
        "wednesday":42,
        "thrusday":40.5,
        "friday":41.5,
        "satruday":40}
for i,j in temp.items():
    if(j>=40 and j<=50):
        print(i,j)
```

```
sunday 40
tuesday 41
wednesday 42
thrusday 40.5
friday 41.5
satruday 40
```

```
L1=["hello","fgg"]
L1.append(["gdsg"])
print(L1)
```

```
['hello', 'fgg', ['gdsg']]
```

practical-30 Write a program to perform below operations on dictionary: □ Create a dictionary. □ Print dictionary items. □ Add/remove key-value pair in/from a dictionary. □ Check whether a key exists in a dictionary. □ Iterate through a dictionary. □ Concatenate multiple dictionaries.

```

A={'Name':'vatsal','Age':17}
print("Element in dictionary are:")
print(A)
A['Address']='adajan'
print("Add element:")
print(A)
A.pop('Address')
print("Remove element:")
print(A)
key='Name'
if key in A.keys():
    print("Present in dictionary")
    print("value =", A[key])
else:
    print("Not present")
print("Iteration of Dictionary:")
for x in A:
    print(x)
A_1={'City':'Surat','Address':'adajan'}
A.update(A_1)
print('Updated dictionary:')
print(A)
d_1 = {'A': 15, 'B': 10, 'C' : 12 }
d_2 = {'E': 18, 'B': 20, 'D' : 16 }
d_1.update(d_2)
print('Updated dictionary:')
print(d_1)

```

```

Element in dictionary are:
{'Name': 'vatsal', 'Age': 17}
Add element:
{'Name': 'vatsal', 'Age': 17, 'Address': 'adajan'}
Remove element:
{'Name': 'vatsal', 'Age': 17}
Present in dictionary
value = vatsal
Iteration of Dictionary:
Name
Age
Updated dictionary:
{'Name': 'vatsal', 'Age': 17, 'City': 'Surat', 'Address': 'adajan'}
Updated dictionary:
{'A': 15, 'B': 20, 'C': 12, 'E': 18, 'D': 16}

```

practical: 31 Write a program that is given a dictionary containing the average daily temperature for each day of the week, and prints all the days on which the average temperature was between 40 and 50 degrees.

```

weekDay = {'sunday':38,
            'monday':41,
            'Tuesday':45,
            'wednesday':39,
            'friday':42,
            'saturday':42}
for key,value in weekDay.items():
    if(value>=40 and value<=50):
        print(key,value)

```

```

monday: 41
Tuesday: 45
friday: 42
saturday: 42

```

practical- 32 Write a Python script to check whether a given key already exists in a dictionary. If it does, drop that key from the dictionary. If it does not, add that key and corresponding value to the dictionary

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
key = 'b'
value = 5

if key in my_dict:

    del my_dict[key]
    print(f"Key '{key}' found and removed.")
else:

    my_dict[key] = value
    print(f"Key '{key}' not found. Added with value {value}.")

print("Updated dictionary:", my_dict)
```

```
Key 'b' found and removed.
Updated dictionary: {'a': 1, 'c': 3}
```

Start coding or [generate](#) with AI.

practical:- 33 Write a program that defines a function (shuffle) to scramble a list into a random order, like shuffling a deck of cards

```
import random
list1 = [9,1,7,3,0,1,8,1,9,2]

def shuffle(list1):
    random.shuffle(list1)
    return list1

shuffle(list1)
```

```
[9, 9, 8, 1, 2, 1, 7, 3, 1, 0]
```

practical:- 34 Write a program that defines a function to return a new list by eliminating the duplicate values in the list.

```
l = [10,10,11,13,8,8,1,2,1,2]
list1 = []
def duplicate(l):
    for i in l:
        if i not in list1:
            list1.append(i)
    return list1
duplicate(l)
```

```
[10, 11, 13, 8, 1, 2]
```

practical:-35 Write a program that lets the user enter the loan amount, number of years, and interest rate, and defines a function to calculate monthly EMI, total payment and display the amortization schedule for the loan.

```
def loan():
    p=float(input("Enter the Amount of Loan:"))
    i=int(input("Enter the Interest:"))
    y=int(input("Enter Total No. of Year:"))
    total_interest=round(p*y*i)/100
    total_amount=round(p+total_interest)
    emi=round(total_amount/(y*12))
    print("interest:",total_interest)
    print("amount:",total_amount)
    print("EMI:",emi)
    print("Principal EMI Interest Balance")
    balance=total_amount-emi
    print(total_amount,"\t",emi,"\t",i,"\t",balance)
    for i in range(1,y*12):
        total_amount=balance
        balance=total_amount-emi
        print(total_amount,"\t",emi,"\t",i,"\t",balance)

loan()
```

```
Enter the Amount of Loan:50000
Enter the Interest:7
Enter Total No. of Year:1
interest: 3500.0
amount: 53500
EMI: 4458
Principal EMI Interest Balance
53500 4458 7 49042
49042 4458 1 44584
44584 4458 2 40126
40126 4458 3 35668
35668 4458 4 31210
31210 4458 5 26752
26752 4458 6 22294
22294 4458 7 17836
17836 4458 8 13378
13378 4458 9 8920
8920 4458 10 4462
4462 4458 11 4
```

Practical:- 36 Write a program to print Fibonacci sequence up to n numbers using recursion. Fibonacci sequence is defined as below: *Fibonacci Sequence*= 1 1 2 3 5 8 13 21... *th where n term* $x = x + x_{n-1} x_{n-2}$

```
# Function to calculate the Fibonacci number using recursion
def fibonacci(n):
    if n == 1 or n == 2:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

# Function to print Fibonacci sequence up to 'n' numbers
def print_fibonacci_sequence(n):
    for i in range(1, n+1):
        print(fibonacci(i), end=" ")

# Example usage
n = int(input("Enter the number of terms: "))
print("Fibonacci Sequence:")
print_fibonacci_sequence(n)
```

```
Enter the number of terms: 7
Fibonacci Sequence:
1 1 2 3 5 8 13
```

Practical:- 37 Write a program that defines a function to determine whether input number n is prime or not. A positive whole number $n > 2$ is prime, if no number between 2 and n (inclusive) evenly divides n . If n is not prime, the program should quit as soon as it finds a value that evenly divides n .

```
def is_prime(n):
    # If the number is less than 2, it's not prime
    if n <= 1:
        return False

    # Check divisibility starting from 2 to sqrt(n)
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False # If n is divisible by i, it's not prime

    # If no divisors found, the number is prime
    return True

# Example usage
n = int(input("Enter a number: "))
if is_prime(n):
    print(f"{n} is a prime number.")
else:
    print(f"{n} is not a prime number.")
```

```
Enter a number: 1
1 is not a prime number.
```

Practical:- 38 Write a program that defines a function to find the GCD of two numbers using the algorithm below. The greatest common divisor (GCD) of two values can be computed using Euclid's algorithm. Starting with the values m and n , we repeatedly apply the formula: $n, m = m, n \% m$ until m is 0. At that point, n is the GCD of the original m and n (Use Recursion).

practical:-39 Write a program that defines functions (mean and deviation), that computes mean and standard deviation of given numbers. The formula for the mean and standard deviation of n numbers is given as: $\text{mean} = \sum_{i=1}^n X_i = X_1 + X_2 + \dots + X_n$

deviation = $\sqrt{\sum_{i=1}^n (X_i - \text{mean})^2 / (n-1)}$

$\sum_{i=1}^n (X_i - \text{mean})^2 / (n-1)$

Practical :- 40

```
def mean(number):
    return sum(number)/len(number)

def deviation(number):
    mean_val = mean(number)
    variances = [(x - mean_val)**2 for x in number]
    return(sum(variances)/(len(number)-1))*0.5

number = [1,2,3,4,5]

mean_val = mean(number)
dev_val = deviation(number)

print('mean of the given number is ',mean_val)
print('standard deviation of the given number is ',dev_val)
```

```
mean of the given number is  3.0
standard deviation of the given number is  1.5811388300841898
```

practical:- 41 Write a program to print the dates of all the Sundays in a given year.

```
import datetime

year = int(input("Enter the Year : "))

first_sunday = datetime.date(year,1,1)

if first_sunday.isoweekday() != 7:
    first_sunday = first_sunday + datetime.timedelta(7-first_sunday.isoweekday())

while first_sunday.year == year:
    print(first_sunday)
    first_sunday=first_sunday + datetime.timedelta(days=7)
```

```
Enter the Year : 2024
2024-01-07
2024-01-14
2024-01-21
2024-01-28
2024-02-04
2024-02-11
2024-02-18
2024-02-25
2024-03-03
2024-03-10
2024-03-17
2024-03-24
2024-03-31
2024-04-07
2024-04-14
2024-04-21
2024-04-28
2024-05-05
2024-05-12
2024-05-19
2024-05-26
2024-06-02
2024-06-09
2024-06-16
2024-06-23
2024-06-30
2024-07-07
2024-07-14
2024-07-21
2024-07-28
2024-08-04
2024-08-11
2024-08-18
```

2024-08-25
2024-09-01
2024-09-08
2024-09-15
2024-09-22
2024-09-29
2024-10-06
2024-10-13
2024-10-20
2024-10-27
2024-11-03
2024-11-10
2024-11-17
2024-11-24
2024-12-01
2024-12-08
2024-12-15
2024-12-22
2024-12-29

practical:- 42 Write a program to display a graph for ReLU (Rectified Linear Unit) function. ReLU function is defined as below: $y = \max(0, x)$ Consider the range of x from -5 to 5.

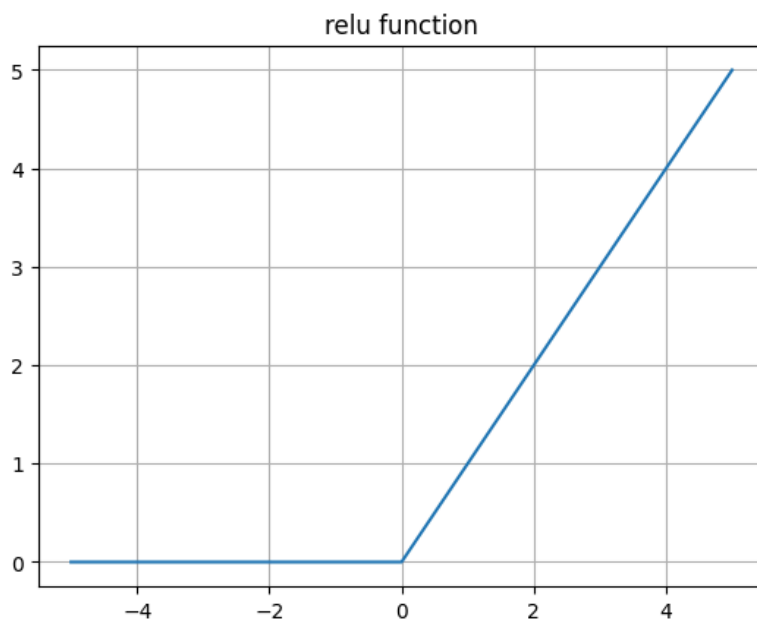
```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(-5,6)

def ReLU(x):
    return np.maximum(0,x)

y=ReLU(x)

plt.title('relu function')
plt.plot(x,y)
plt.grid()
plt.show()
```



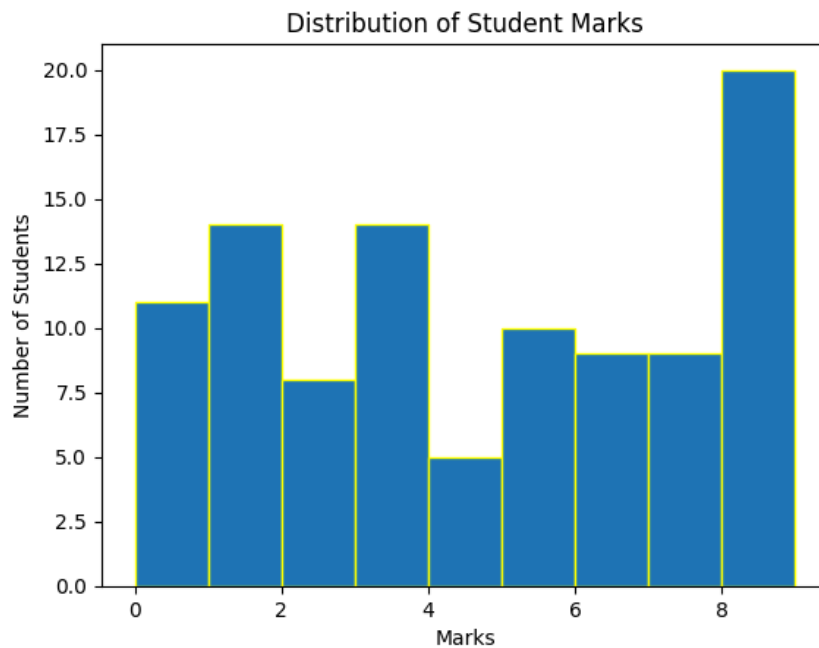
practical:- 43 Write a program to create a list representing the results of 100 students in a test, where each element represents a student's marks (between 0 to 10), and display a histogram for the result.

```
import random
import matplotlib.pyplot as plt
```

```
# Generate a list of 100 student marks between 0 and 10
student_marks = [random.randrange(0, 10) for _ in range(100)]

# Display a histogram
plt.hist(student_marks, range(0, 10), edgecolor='yellow')
plt.xlabel('Marks')
plt.ylabel('Number of Students')
plt.title('Distribution of Student Marks')

plt.show()
```



practical:- 44

Create a user defined module with simple functions for: addition, subtraction, multiplication, division, modulo, square, factorial. Write a program to import the module and access functions defined in the module.

```
%writefile math_operations4.py
import math
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    return x / y

def modulo(x, y):
    return x % y

def square(x):
    return x ** 2
```

```
def factorial(x):
    return math.factorial(x)
```

Writing math_operations4.py

```
import math_operations4 as mo

def main():
    # Test the functions in the module
    print("Addition: ", mo.add(5, 3))
    print("Subtraction: ", mo.subtract(5, 3))
    print("Multiplication: ", mo.multiply(5, 3))
    print("Division: ", mo.divide(5, 3))
    print("Modulo: ", mo.modulo(5, 3))
    print("Square: ", mo.square(5))
    print("Factorial: ", mo.factorial(5))

# Execute the main function
main()
```

```
Addition: 8
Subtraction: 2
Multiplication: 15
Division: 1.6666666666666667
Modulo: 2
Square: 25
Factorial: 120
```

practical:- 45 Write a program to check whether a given string is palindrome or not.

```
my_str = str(input("enter the string:"))

rev_str = reversed(my_str)

if list(my_str) == list(rev_str):
    print("The string is palindrome.")
else:
    print("The string is not palindrome.")
```

```
enter the string:121
The string is palindrome.
```

practical:- 46

Write a program to read a string containing letters, each of which may be in either uppercase or lowercase, and return a tuple containing the number of vowels and consonants in the string.

```
a=input('enter a string:')
consonant=0
vowel=0
for i in a:
    if(i=="A" or i=="E" or i=="I" or i=="O" or i=="U" or i=="a" or i=="e" or i=="i" or i=="o" or i=="u"):
        vowel=vowel+1
    else:
        consonant=consonant+1
t1=tuple(('consonant:',consonant,"vowel:",vowel))
print(t1)
```

```
enter a string:bhumika
('consonant:', 4, 'vowel:', 3)
```

Practical-47 Write a program to read a date in the format DD/MM/YYYY and print the same date in MM- DD-YYYY format.

```
def date():
    d = input("Enter the Date in DD/MM/YYYY format = ")
    a = d.split("/") # Corrected to split by "/"
    print("Date in format MM-DD-YYYY = {}-{}-{}".format(a[1], a[0], a[2]))

date()
```

```
Enter the Date in DD/MM/YYYY format = 4/09/2005
Date in format MM-DD-YYYY = 09-4-2005
```

Practical-48

Write a program that checks whether two words are anagrams. Two words are anagrams if they contain the same letters. For example, silent and listen are anagrams.

```
def anagram():
    w1=input("enter word:")
    w2=input("enter word:")
    a=w1.lower()
    b=w2.lower()
    x=sorted(a)
    y=sorted(b)
    if x==y:
        print(w1,"and",w2,"ARE ANAGRAM WORD")
    else:
        print(w1,"and",w2,"ARE NOT ANAGRAM WORD")
anagram()
```

```
enter word:bmr
enter word:bvh
bmr and bvh ARE NOT ANAGRAM WORD
```

Practical-49

Write a program that allows users to enter six- digit RGB color codes and converts them into base 10. In this format, the first two hexadecimal digits represent the amount of red, the second two the amount of green, and the last two the amount of blue. For example: If a user enters FF6347, then the output should be Red (255), Green (99) and Blue (71))

```
def RGB():
    code=input("ENTER THE RGB COLOR CODE=")
    red=int(code[0:2],16)
    green=int(code[2:4],16)
    blue=int(code[4:6],16)
    print("red=",red,"green=",green,"blue=",blue)

RGB()
```

```
ENTER THE RGB COLOR CODE=ff6347
red= 255 green= 99 blue= 71
```

Practical-50

Numerologists claim to be able to determine a person's character traits based on the "numeric value" of a name. The value of a name is determined by summing up the values of the letters of the name, where "a" is 1 "b" is 2 "c" is 3 and so

on up to "z" being 26. For example, the name "Python" would have the value $16 + 25 + 20 + 8 + 15 + 14 = 98$. Write a program that calculates the numeric value of a name provided as input.

```
def name_sum():
    name=input("ENTER THE NAME=")
    sum=0
    for letter in name.lower():
        print(letter,"=",ord(letter)-96)
        sum=sum+ord(letter)-96
        print("letter's sum=",sum)

name_sum()
```

```
ENTER THE NAME=python
p = 16
letter's sum= 16
y = 25
letter's sum= 41
t = 20
letter's sum= 61
h = 8
letter's sum= 69
o = 15
letter's sum= 84
n = 14
letter's sum= 98
```

```
a="vatsal"
x=a.ljust(11,"o")
print(x)
```

```
vatsalooooo
```

```
ord("c")-96
```

```
3
```

Practical :- 51

Write a program to perform the below operations on files: • Create a text file and write a string to it. • Read an entire text file. • Read a text file line by line. • Write a string to a file. • Write a list of strings to a file. • Count the number of lines, words in a file.

```
%%writefile "test1.txt"
hello
this is my first lines
this is my Second lines
how are you
```

```
Overwriting test1.txt
```

```
f=open("test1.txt","r")
print(f.read())
f.close()
```

```
hello
this is my first lines
this is my Second lines
how are you
```

```
f=open("test1.txt","r")
b=f.readlines()
print(b)
f.close()
```

```
['hello\n', 'this is my first lines\n', 'this is my Second lines\n', 'how are you\n']
```

```
f=open("test1.txt","a")
f.write("hello world 22")
f=open("test1.txt","r")
print(f.read())
f.close()
```

```
hello
this is my first lines
this is my Second lines
how are you
hello world 22hello world 22
```

```
l=["hi ","i am Bhumika ","how are you?","no"]
f=open("test1.txt","a")
for i in l:
    f.write(i)
f=open("test1.txt")
print(f.read())
f.close()
```

```
hello
this is my first lines
this is my Second lines
how are you
hello world 22hello world 22hi i am Bhumika how are you?no
```

```
f=open("test1.txt")
c=f.readlines()
a=[]
b=[]
for i in c:
    b=i.split()
    a.extend(b)
print("number of lines",len(c))
print("number of words",len(a))
```

```
number of lines 5
number of words 25
```

Practical :- 52

Write a program that reads a text file and counts the occurrences of each alphabet in the file. The program should prompt the user to enter the filename.

```
from string import ascii_lowercase as alc
files=input("enter file name=")
f=open(files,"r")
a=f.read()

for i in alc:
    x=a.count(str(i))
    print(i,"is featured for ",x,"number of times")
f.close()
```

```
enter file name=test1.txt
a is featured for 4 number of times
```

```

b is featured for 0 number of times
c is featured for 1 number of times
d is featured for 3 number of times
e is featured for 8 number of times
f is featured for 1 number of times
g is featured for 0 number of times
h is featured for 9 number of times
i is featured for 10 number of times
j is featured for 0 number of times
k is featured for 1 number of times
l is featured for 10 number of times
m is featured for 4 number of times
n is featured for 4 number of times
o is featured for 11 number of times
p is featured for 0 number of times
q is featured for 0 number of times
r is featured for 5 number of times
s is featured for 7 number of times
t is featured for 3 number of times
u is featured for 3 number of times
v is featured for 0 number of times
w is featured for 4 number of times
x is featured for 0 number of times
y is featured for 4 number of times
z is featured for 0 number of times

```

Practical :- 53

Write a program that reads a text file and displays all the numbers found in the file.

```

files=input("enter file name :")
f=open(files,"r")
a=f.read()
for i in range(10):
    x=a.count(str(i))
    print(i,"is featured for",x,"number of times in a string")

```

```

enter file name :test.txt
0 is featured for 0 number of times in a string
1 is featured for 0 number of times in a string
2 is featured for 0 number of times in a string
3 is featured for 0 number of times in a string
4 is featured for 0 number of times in a string
5 is featured for 0 number of times in a string
6 is featured for 0 number of times in a string
7 is featured for 0 number of times in a string
8 is featured for 0 number of times in a string
9 is featured for 0 number of times in a string

```

```

a="h e l l o 1 2 "
x=a.find("l")
print(x)

```

4

```
%%writefile "test6.txt"
```

```

Gujarat Technological.
fdofsjl lorem50

```

Writing test6.txt

```

f=open("test6.txt","r", encoding='utf-8')
c=f.read()
a=[]
b=[]
y=0

```

```

sent_length=0
a=c.split()
for i in a:
    y+=len(i)
num_of_sentence=c.split(".")
for sentence in num_of_sentence:
    sent_length+= len(sentence)
avg_wordlen=y/len(a)
avg_sentencelen=sent_length/len(num_of_sentence)

print("Aver word length : ",avg_wordlen)
print("Aver sentence length : ",avg_sentencelen)
f.close()

```

Aver word length : 8.75
Aver sentence length : 19.5

```

%%writefile "hello.txt"
hello world

```

Writing hello.txt

```

%%writefile "hello1.txt"
Sky is the Limit

```

Writing hello1.txt

```

H1=open("hello.txt","r")
H2=open("hello1.txt","r")
s1=H1.read()
s2=H2.read()
print("Contents of file 1 are:",s1)
print("Contents of file 2 are:",s2,"\n")
s1=s1.replace(" ","")
s2=s2.replace(" ","")
op=""
size = len(s1) if len(s1) > len(s2) else len(s2)
for i in range(size):
    if i<len(s1):
        op=op+s1[i]
    if i<len(s2):
        op=op+s2[i]
print("interelated file contents are: \n\n",op)

```

Contents of file 1 are: hello world

Contents of file 2 are: Sky is the Limit

interelated file contents are:

hSeklylioswtohrelldi
mit

Close

****Thank You For Your Time****

Thank You For Your Time

```

import requests
import fontstyle
import bs4

```

```
headers = {  
    'User-Agent': 'Mozilla/5.0 (Windows NT 11.0; Win64; x64) AppleWebKit/  
}  
  
print()  
topic = fontstyle.apply('Scraping Google Results in PYTHON', 'bold/WHITE/CYAN_  
print(topic)  
print()  
  
print()  
query = input("Enter your Query : ")  
print()
```